Введение

Вычислительный интеллект, лекция 1

Сергей Савин

2026

Мотивация

Многие современные методы (например, планирование траекторий, оценка состояния, синтез управления) опираются на инструменты численной оптимизации. В этом курсе мы изучим численную оптимизацию с акцентом на выпуклые методы.

Чего мы хотим?

Перейти от принципа "надеюсь, это сработает"к глубокому пониманию математики и сценариев использования этих инструментов.

Зачем нам это?

Это должно позволить нам решать гораздо более широкий круг задач, причем более эффективно.

Мотивирующий пример

У нас есть следующая задача: найти такой $\mathbf{x}$, который минимизирует $\mathbf{x}^\top \mathbf{M}\mathbf{x}$, при условии $\mathbf{C}\mathbf{x} = \mathbf{y}$. Другими словами:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \mathbf{x}^\top \mathbf{M}\mathbf{x}, \\ \text{subject to:} \quad & \mathbf{C}\mathbf{x} = \mathbf{y}. \end{aligned} \tag{1}$$

Более конкретно:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 5 & 0 \\ 1 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \\ \text{subject to:} \quad & \begin{bmatrix} 1 & 7 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 1. \end{aligned} \tag{2}$$

Как нам ее решить?

Мотивирующий пример

fmincon

Очень популярное решение — использовать солвер локальной оптимизации общего назначения, например **fmincon** из MATLAB. Вот одно из возможных решений:

```
M = [1 0 1; 0 5 0; 1 0 3];
C = [1 7 2];
y = 1;
fnc = @(x) x'Mx;
con = @(x) deal([], C*x-y);
x = fmincon(fnc, zeros(3, 1), [],[],[],[],[],[],
con)
```

Среднее время решения составляет **2.5** мс (это зависит от многих факторов, поэтому рассматривайте это только как относительную информацию). Решение:
$\mathbf{x} = \begin{bmatrix} 0.0442 & 0.1239 & 0.0442 \end{bmatrix}$.

Мотивирующий пример

quadprog

Более изощренный, но все еще очень прямолинейный подход — использовать специализированный решатель для данного класса задач — `quadprog` из MATLAB. Вот решение:

```
M = [1 0 1; 0 5 0; 1 0 3];
C = [1 7 2];
y = 1;

x = quadprog(2*M,[],[],[],C,y)
```

Среднее время решения составляет **0.3** мс, что на порядок меньше, чем с `fmincon`.

Мотивирующий пример

Решение на основе SVD

Мы можем использовать алгебраическое решение, основанное на SVD-разложении (через ядро и псевдообратную матрицу), следующим образом:

```
M = [1 0 1; 0 5 0; 1 0 3];
C = [1 7 2];
y = 1;

tol = 10^(-5);
[P, N] = pinv_null(C, tol);
x = ( eye(3) - N((N'MN) \ (N'M)) ) * P*y
```

Где `pinv_null` — это функция, объединяющая `pinv` и `null`, полученная из одного SVD-разложения.
Среднее время решения составляет **0.016** мс, что $\sim$ в 20 раз быстрее, чем `quadprog`, и $\sim$ в 150 раз быстрее, чем `fmincon`.

Мотивирующий пример

Левое матричное деление

Мы можем использовать специализированный инструмент для решения линейных уравнений — `mldivide`, который в нашем случае применит LU-разложение (размер матрицы меньше 16x16). Вот решение:

```
T1 = M \ C';
t2 = (C*T1) \ y;
x = T1*t2;
```

Среднее время решения составляет **0.0065** мс, что в **380** раз быстрее, чем `fmincon`, в **45** раз быстрее, чем `quadprog`, и в **2.5** раза быстрее, чем SVD.

Мотивирующий пример

Решение на основе CVX

Наконец, мы можем задействовать один из самых мощных инструментов выпуклой оптимизации с удобным для пользователя стилем кодирования — CVX:

```
M = [1 0 1; 0 5 0; 1 0 3];
C = [1 7 2];
y = 1;
cvx_begin
variables x(3);
minimize( x' * M * x );
subject to
C*x == y;
cvx_end
```

Однако мы увидим, что накладные расходы на вызов решателя для этой задачи чрезмерны. Среднее время решения составляет **219** мс, что $\sim$ в 90 раз медленнее, чем `fmincon`.

Слайды доступны на Github:

github.com/SergeiSa/Convex-Optimization